

Overhead de Operações Criptográficas em Bancos de Dados Distribuídos

Beatriz Viana Costa¹, Lys Katherine Park Kang¹,
Raphaella Brandão Jacques¹, Stephanie Maria Braga¹

¹Instituto de Matemática e Estatística (IME)
Universidade de São Paulo (USP) – São Paulo, SP – Brasil

Abstract. *As distributed systems increasingly store sensitive data, cryptographic techniques become essential to guarantee confidentiality and integrity. However, these techniques introduce performance costs that are amplified in distributed settings, where inter-node communication is already a natural bottleneck. This work investigates the impact of cryptographic operations on the performance of distributed databases, combining a literature review with practical experiments. We configure a distributed database and measure latency and throughput under different scenarios, with and without encryption (in transit and at rest), in order to quantify the introduced overhead and discuss the trade-offs between security and performance.*

Resumo. *À medida que sistemas distribuídos passam a armazenar volumes crescentes de dados sensíveis, técnicas criptográficas tornam-se indispensáveis para garantir confidencialidade e integridade. Contudo, essas técnicas introduzem custos de desempenho que se tornam ainda mais relevantes em ambientes distribuídos, nos quais a comunicação entre nós já representa um gargalo natural. Este trabalho investiga o impacto de operações criptográficas sobre o desempenho de bancos de dados distribuídos, combinando uma revisão da literatura com experimentos práticos. Configuramos um banco de dados distribuído e medimos latência e vazão sob diferentes cenários, com e sem criptografia (em trânsito e em repouso), a fim de quantificar o overhead introduzido e discutir os trade-offs entre segurança e desempenho.*

1. Introdução

A crescente adoção de bancos de dados distribuídos para o armazenamento de grandes volumes de dados sensíveis torna a proteção criptográfica um requisito central de projeto [Fuller et al. 2017]. Entretanto, garantir confidencialidade e integridade tem um custo: operações criptográficas consomem CPU e introduzem latência adicional, efeito agravado em sistemas distribuídos, nos quais a comunicação entre nós já constitui um gargalo natural [Gilbert and Lynch 2002].

Este trabalho tem como objetivo quantificar o overhead introduzido por operações criptográficas sobre o desempenho de bancos de dados distribuídos e discutir o trade-off entre segurança e desempenho. Especificamente, buscamos responder:

- **PP1.** Qual o impacto da criptografia *em trânsito* (TLS) sobre latência e vazão?
- **PP2.** Qual o impacto da criptografia *em repouso* (TDE)?
- **PP3.** Como esse overhead varia com a carga, o número de nós e o perfil de leitura/escrita?

O restante do artigo está organizado como segue. A Seção 2 apresenta os conceitos fundamentais. A Seção 3 discute o estado da arte. A Seção 4 detalha a metodologia experimental. A Seção 5 apresenta os resultados, discutidos na Seção 6. A Seção 7 conclui o trabalho.

2. Conceitos Fundamentais

Para compreender o impacto de operações criptográficas em bancos de dados distribuídos, é necessário primeiro entender como esses sistemas funcionam, quais mecanismos de criptografia estão disponíveis e como medimos seu desempenho. Esta seção apresenta esses três pilares.

2.1. Bancos de Dados Distribuídos

Um **Banco de Dados Distribuído** (BDD) é uma coleção de bancos de dados logicamente inter-relacionados, fisicamente dispersos por uma rede de computadores e gerenciados por um Sistema de Gerenciamento de Banco de Dados Distribuído (SGBDD) [Elmasri and Navathe 2016]. A motivação para distribuir dados inclui maior disponibilidade, melhor desempenho por proximidade geográfica e escalabilidade horizontal. Contudo, essa distribuição introduz desafios que não existem em sistemas centralizados – em particular, a necessidade de comunicação constante entre nós, que é justamente o canal sobre o qual a criptografia em trânsito atua.

2.1.1. Fragmentação e Replicação

Para distribuir os dados entre os nós, um BDD utiliza duas técnicas complementares: fragmentação e replicação [Elmasri and Navathe 2016].

A **fragmentação** (também chamada de *sharding* ou particionamento) consiste em dividir uma relação em partes menores, cada uma alocada em um nó diferente. Há três abordagens: a *fragmentação horizontal* divide as tuplas segundo algum critério de seleção (por exemplo, por região geográfica); a *fragmentação vertical* distribui os atributos da relação, como em uma operação de projeção; e a *fragmentação mista* combina ambas as estratégias. A escolha da estratégia influencia diretamente o volume de comunicação entre nós – uma consulta que precisa recombinar fragmentos dispersos gera tráfego de rede, e portanto mais dados a serem cifrados quando TLS está habilitado.

A **replicação**, por sua vez, mantém cópias dos dados em múltiplos nós. Isso melhora a disponibilidade (se um nó falha, outro responde) e o desempenho de leitura (requisições podem ser atendidas localmente). Em contrapartida, escritas precisam ser propagadas para todas as réplicas, gerando tráfego adicional de sincronização. Em um cenário com TLS, cada mensagem de sincronização entre réplicas também passa por cifragem, amplificando o overhead proporcionalmente ao grau de replicação.

2.1.2. Comunicação e Processamento de Consultas

Em um BDD, o processamento de consultas difere fundamentalmente do caso centralizado: o otimizador precisa considerar não apenas o custo de I/O e CPU, mas também o **custo de comunicação** – isto é, o volume de dados transferidos pela rede entre

os nós [Elmasri and Navathe 2016]. Estratégias como a semi-junção (*semi-join*) existem justamente para reduzir esse tráfego. Esse custo de comunicação é o ponto central deste trabalho: quando habilitamos TLS, cada byte transmitido entre nós precisa ser cifrado na origem e decifrado no destino, adicionando latência a cada troca de mensagem.

Além disso, para garantir a atomicidade de transações que envolvem múltiplos nós, BDDs utilizam protocolos de coordenação como o *Two-Phase Commit* (2PC), em que um coordenador troca mensagens com todos os participantes para decidir se a transação será confirmada ou abortada [Elmasri and Navathe 2016]. Protocolos de consenso mais modernos, como Raft [Ongaro and Ousterhout 2014], são utilizados em sistemas como Spanner [Corbett et al. 2012]. Todos esses protocolos dependem de múltiplas rodadas de comunicação, o que os torna sensíveis ao overhead de criptografia em trânsito.

2.1.3. Teorema CAP

O teorema CAP [Gilbert and Lynch 2002] formaliza um trade-off fundamental: em um sistema distribuído sujeito a partições de rede, é impossível garantir simultaneamente *Consistência* (todos os nós veem os mesmos dados), *Disponibilidade* (toda requisição recebe resposta) e *Tolerância a Partições*. Como partições são inevitáveis na prática, sistemas precisam optar entre priorizar consistência (modelo CP, como Spanner) ou disponibilidade (modelo AP, como Cassandra). Essa escolha arquitetural determina a quantidade e o padrão de comunicação entre nós, influenciando diretamente o quanto o overhead do TLS se manifesta no desempenho global do sistema.

2.2. Técnicas Criptográficas em Bancos de Dados

A proteção de dados em um banco de dados pode ocorrer em três momentos distintos, de acordo com o estado em que os dados se encontram [Force 2020].

2.2.1. Criptografia em Repouso (*at rest*)

Quando os dados estão armazenados em disco, a criptografia em repouso protege contra acessos não autorizados ao meio físico (roubo de disco, acesso indevido a arquivos do sistema operacional). A abordagem mais comum é a *Transparent Data Encryption* (TDE), que cifra as páginas de dados e os arquivos de log de forma transparente para a aplicação – ou seja, o banco realiza a cifragem e decifragem automaticamente nas operações de I/O, sem alterar as consultas [Force 2020].

O algoritmo predominante é o AES (*Advanced Encryption Standard*), um cifrador de bloco simétrico padronizado pelo NIST [National Institute of Standards and Technology (NIST) 2001], com chaves de 128, 192 ou 256 bits. A granularidade pode variar: criptografia de todo o disco, por *tablespace*, por tabela ou mesmo por coluna individual. O overhead da criptografia em repouso incide sobre cada operação de leitura e escrita em disco, pois os dados precisam ser decifrados ao serem carregados para a memória e cifrados novamente ao serem persistidos.

2.2.2. Criptografia em Trânsito (*in transit*)

A criptografia em trânsito protege os dados enquanto trafegam pela rede – seja entre um cliente e o servidor, seja entre os nós internos do cluster distribuído. O protocolo padrão é o TLS (*Transport Layer Security*) [Rescorla 2018], atualmente na versão 1.3.

O custo de desempenho do TLS pode ser decomposto em dois componentes:

1. **Handshake:** no estabelecimento da conexão, cliente e servidor negociam parâmetros criptográficos utilizando criptografia assimétrica (RSA ou curvas elípticas). É uma operação computacionalmente cara, mas realizada apenas uma vez por conexão (ou por reconexão).
2. **Cifragem simétrica contínua:** após o handshake, todos os dados são cifrados com algoritmos simétricos como AES-GCM ou ChaCha20-Poly1305. O custo por pacote é baixo individualmente, mas em cenários de alta vazão (muitas operações por segundo ou grandes volumes de dados) o impacto acumula.

Em um banco de dados distribuído, o TLS pode ser habilitado tanto na comunicação cliente–servidor quanto na comunicação interna entre os nós do cluster (*intra-cluster*). Neste segundo caso, o impacto tende a ser mais expressivo, pois protocolos de replicação e consenso geram tráfego constante entre os nós.

2.2.3. Criptografia em Uso (*in use*)

Uma linha de pesquisa mais recente busca permitir computações diretamente sobre dados cifrados, eliminando a necessidade de decifrá-los. As abordagens incluem: criptografia determinística e que preserva ordem (OPE) [Boldyreva et al. 2009], que viabilizam consultas de igualdade e intervalos sobre dados cifrados, como explorado pelo sistema CryptDB [Popa et al. 2011]; criptografia totalmente homomórfica (FHE) [Gentry 2009], que permite computações arbitrárias mas com overhead ainda proibitivo para cargas transacionais; e soluções baseadas em enclaves seguros (*trusted execution environments*), como o Always Encrypted [Antonopoulos et al. 2020]. Embora não sejam o foco dos nossos experimentos, essas técnicas representam o estado da arte em proteção de dados e servem como horizonte para trabalhos futuros.

2.2.4. Integridade

Complementando a confidencialidade, mecanismos de integridade garantem que os dados não foram corrompidos ou adulterados. Técnicas como *hashing* criptográfico (SHA-256), *HMAC* (Hash-based Message Authentication Code) e árvores de Merkle permitem detectar alterações tanto em dados armazenados quanto em dados em trânsito [Force 2020]. No contexto do TLS, a integridade é garantida implicitamente pelos modos de operação AEAD (*Authenticated Encryption with Associated Data*), como o AES-GCM, que combina cifragem e autenticação em uma única operação.

2.3. Métricas de Desempenho

Para avaliar o impacto das operações criptográficas de forma rigorosa, é necessário definir as métricas que serão observadas [Cooper et al. 2010]:

- **Latência:** o tempo entre o envio de uma requisição e o recebimento da resposta correspondente. Como a distribuição de latências tende a ser assimétrica (com uma “cauda longa”), reportamos não apenas a média, mas também a mediana (p50) e os percentis p95 e p99, que revelam o comportamento nos piores casos.
- **Vazão (*throughput*):** o número de operações completadas por unidade de tempo (operações/segundo). Enquanto a latência captura a experiência individual de cada requisição, a vazão reflete a capacidade agregada do sistema sob carga.
- **Utilização de CPU:** a fração do processamento dedicada a operações criptográficas. Permite distinguir se o overhead observado decorre de cifragem ou de outros fatores (I/O, contenção de rede).
- **Overhead de rede:** o aumento no volume de bytes transmitidos devido aos cabeçalhos e metadados introduzidos pelo TLS (registros TLS, MACs, padding).

Para quantificar o impacto relativo da criptografia, definimos o **overhead** em relação ao cenário *baseline* (sem criptografia):

$$\text{overhead}(\%) = \frac{M_{\text{cripto}} - M_{\text{baseline}}}{M_{\text{baseline}}} \times 100 \quad (1)$$

onde M representa o valor de uma métrica (latência ou vazão) no respectivo cenário. Um overhead positivo em latência indica degradação; um overhead negativo em vazão indica queda na capacidade do sistema.

3. Trabalhos Relacionados

4. Metodologia

4.1. Ambiente Experimental

4.2. Carga de Trabalho

4.3. Cenários Avaliados

Tabela 1. Cenários de criptografia avaliados.

Cenário	Em trânsito (TLS)	Em repouso (TDE)
C0 (baseline)	não	não
C1	sim	não
C2	não	sim
C3	sim	sim

4.4. Protocolo de Medição

5. Experimentos e Resultados

6. Discussão

7. Conclusão e Trabalhos Futuros

Referências

Antonopoulos, P., Arasu, A., Singh, K. D., Eguro, K., Gupta, N., Jain, R., Kaushik, R., Kodavalla, H., Kossmann, D., Ramamurthy, R., Szymaszek, J., Trimmer, J., Vaswani,

- K., Venkatesan, R., and Zwillig, M. (2020). Azure SQL database always encrypted. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1511–1525. ACM.
- Boldyreva, A., Chenette, N., Lee, Y., and O’Neill, A. (2009). Order-preserving symmetric encryption. In *Advances in Cryptology – EUROCRYPT 2009*, volume 5479 of *Lecture Notes in Computer Science*, pages 224–241. Springer.
- Cooper, B. F., Silberstein, A., Tam, E., Ramakrishnan, R., and Sears, R. (2010). Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154. ACM.
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J. J., Ghemawat, S., Gubarev, A., Heiser, C., et al. (2012). Spanner: Google’s globally-distributed database. In *Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 261–264. USENIX Association.
- Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, 7 edition.
- Force, J. T. (2020). Security and privacy controls for information systems and organizations. Special Publication SP 800-53 Rev. 5, National Institute of Standards and Technology (NIST). <https://doi.org/10.6028/NIST.SP.800-53r5>.
- Fuller, B., Varia, M., Yerukhimovich, A., Shen, E., Hamlin, A., Gadepally, V., Shay, R., Mitchell, J. D., and Cunningham, R. K. (2017). SoK: Cryptographically protected database search. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 172–191. IEEE.
- Gentry, C. (2009). Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 169–178. ACM.
- Gilbert, S. and Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59.
- National Institute of Standards and Technology (NIST) (2001). Advanced encryption standard (AES). Federal Information Processing Standards Publication FIPS PUB 197, NIST.
- Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, pages 305–319. USENIX Association.
- Popa, R. A., Redfield, C. M. S., Zeldovich, N., and Balakrishnan, H. (2011). CryptDB: Protecting confidentiality with encrypted query processing. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, pages 85–100. ACM.
- Rescorla, E. (2018). The transport layer security (TLS) protocol version 1.3. RFC 8446, Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc8446>.